

# GeCode CSP Pipeline - Integracion con Gecode

Puente Pascal/C++ y Compilacion Monolitica

Proyecto GeCode CSP Pipeline

2026

## Contents

|                                              |           |
|----------------------------------------------|-----------|
| <b>Introduccion a la Integracion</b>         | <b>3</b>  |
| Vision General . . . . .                     | 3         |
| <b>Arquitectura del Puente Pascal/C++</b>    | <b>5</b>  |
| Componentes del Puente . . . . .             | 5         |
| 1. UGecodeBridge.pas (Lado Pascal) . . . . . | 5         |
| 2. gecode_bridge.cpp (Lado C++) . . . . .    | 5         |
| Flujo de Datos . . . . .                     | 5         |
| <b>Estructuras de Datos Compartidas</b>      | <b>6</b>  |
| TCSPVar / Variable . . . . .                 | 6         |
| TCSPConstraint / Constraint . . . . .        | 6         |
| TCSPSolution / Solution . . . . .            | 7         |
| <b>API Exportada desde C++</b>               | <b>8</b>  |
| Funciones Principales . . . . .              | 8         |
| csp_create . . . . .                         | 8         |
| csp_add_constraint . . . . .                 | 8         |
| csp_solve_first . . . . .                    | 8         |
| csp_solve_all . . . . .                      | 9         |
| csp_count_solutions . . . . .                | 9         |
| csp_count_with_constraint . . . . .          | 9         |
| csp_debug_domains . . . . .                  | 9         |
| csp_free . . . . .                           | 10        |
| <b>Tipos de Restricciones Soportadas</b>     | <b>11</b> |
| Restricciones Basicas . . . . .              | 11        |
| Restricciones de Dominio . . . . .           | 11        |
| Restricciones Lineales . . . . .             | 11        |
| Restricciones de Valor Absoluto . . . . .    | 12        |
| Restricciones de Distancia . . . . .         | 12        |
| Restriccion Global . . . . .                 | 12        |
| <b>Compilacion Monolitica</b>                | <b>13</b> |

|                                              |           |
|----------------------------------------------|-----------|
| Modos de Compilacion . . . . .               | 13        |
| Modo Estatico . . . . .                      | 13        |
| Modo Dinamico . . . . .                      | 13        |
| Script build_monolithic.sh . . . . .         | 13        |
| Proceso de Compilacion . . . . .             | 13        |
| Paso 1: Compilar Bridge C++ . . . . .        | 13        |
| Paso 2: Compilar Pascal . . . . .            | 14        |
| Paso 3: Linkeo Final . . . . .               | 14        |
| Verificacion . . . . .                       | 14        |
| <b>Programas de Prueba</b>                   | <b>16</b> |
| 1. TestGecodeBridge . . . . .                | 16        |
| 2. GecodeInfo . . . . .                      | 16        |
| 3. TestComplejo . . . . .                    | 17        |
| Problema 1: SEND + MORE = MONEY . . . . .    | 17        |
| Problema 2: Cuadrado Magico 3x3 . . . . .    | 18        |
| 4. TestIntervalos . . . . .                  | 18        |
| <b>Ejemplos de Uso</b>                       | <b>20</b> |
| Ejemplo 1: Resolver CSP Simple . . . . .     | 20        |
| Ejemplo 2: Todas las Soluciones . . . . .    | 20        |
| Ejemplo 3: Exploracion del Espacio . . . . . | 21        |
| <b>Resumen de la Integracion</b>             | <b>23</b> |
| Ventajas de esta Arquitectura . . . . .      | 23        |
| Limitaciones . . . . .                       | 23        |
| Flujo Completo . . . . .                     | 23        |

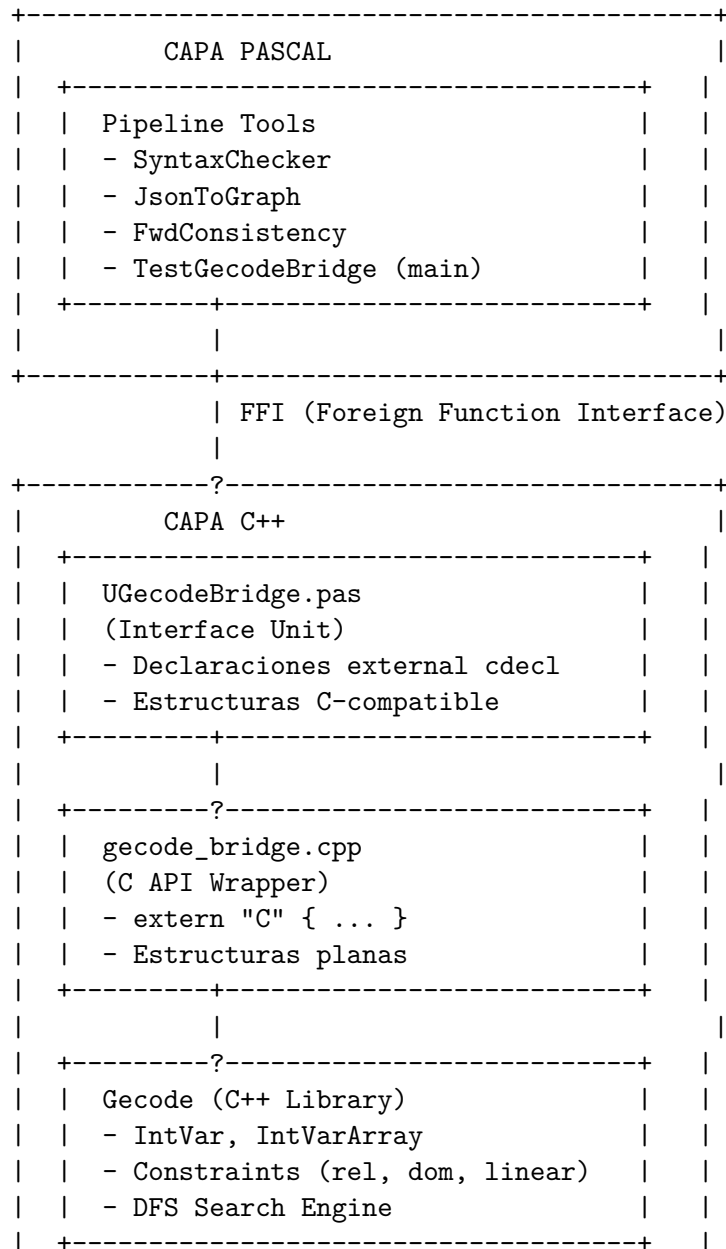
## Introduccion a la Integracion

El GeCode CSP Pipeline integra **tres lenguajes** de forma eficiente para combinar las fortalezas de cada uno:

- **Pascal (Free Pascal)**: Logica de negocio, parsing, pipeline
- **C (MiniMath)**: Funciones matematicas, aritmetica de intervalos
- **C++ (Gecode)**: Resolucion CSP con programacion por restricciones

Esta integracion multi-lenguaje permite crear **ejecutables monoliticos** totalmente standalone que no requieren dependencias externas.

### Vision General



+-----+

# Arquitectura del Puente Pascal/C++

## Componentes del Puente

### 1. UGecodeBridge.pas (Lado Pascal)

**Proposito:** Unit Pascal que declara las funciones C++ exportadas y proporciona helpers para construir estructuras.

**Ubicacion:** /home/rodo/gecode/src/UGecodeBridge.pas

**Caracteristicas:** - Usa `{ $PACKRECORDS C }` para alinear estructuras igual que C/C++ - Usa `{ $L ../obj/gecode_bridge.o }` para linkear el objeto C++ - Declara funciones con `cdecl; external;` (C calling convention) - Proporciona funciones helper para construir estructuras

### 2. gecode\_\_bridge.cpp (Lado C++)

**Proposito:** Wrapper C++ que expone una API C para ser llamada desde Pascal.

**Ubicacion:** /home/rodo/gecode/src/gecode\_\_bridge.cpp

**Caracteristicas:** - Usa `extern "C" { ... }` para exportar simbolos con C linkage - Define clase `CSPModel` que hereda de `Gecode::Space` - Implementa funciones C que crean/manipulan objetos C++ - Usa `#pragma GCC visibility push(hidden)` para ocultar simbolos internos

## Flujo de Datos

Pascal Program

?

Llama: `csp_create(vars, n)`

?

`gecode__bridge.cpp: extern "C" void* csp_create(...)`

?

Crea: `new CSPModel(vars, n)`

?

`Gecode: IntVar, IntVarArray, branch()`

?

Retorna: `void*` (puntero opaco)

?

Pascal: `Model := Pointer` (maneja como handle opaco)

## Estructuras de Datos Compartidas

Las estructuras deben ser **binarias compatibles** entre Pascal y C++. Esto se logra con:

- Pascal: {\$PACKRECORDS C} – alineación C
- C++: Estructuras POD (Plain Old Data) sin virtual tables

### TCSPVar / Variable

Pascal (UGecodeBridge.pas):

```
type
  TCSPVar = record
    Name      : array[0..63] of Char;
    MinDomain : LongInt;
    MaxDomain : LongInt;
  end;
  PCSPVar = ^TCSPVar;
```

C++ (gecode\_bridge.cpp):

```
struct Variable {
    char name[64];
    int  min_domain;
    int  max_domain;
};
```

Uso:

```
var
  V: TCSPVar;
begin
  V := CSPMakeVar('x', 1, 10); // Helper Pascal
  Model := csp_create(@V, 1);  // Llamada a C++
end;
```

### TCSPConstraint / Constraint

Pascal (UGecodeBridge.pas):

```
type
  TCSPConstraint = record
    CType      : LongInt;           { CT_* }
    Var1       : array[0..63] of Char;
    Var2       : array[0..63] of Char;
    Constant   : LongInt;
    Lo, Hi     : LongInt;
    LoOpen, HiOpen : ByteBool;
    SetVals    : array[0..99] of LongInt;
    SetSize    : LongInt;
    LinVars    : array[0..19, 0..63] of Char;
    LinCoefs   : array[0..19] of LongInt;
```

```

LinNVars : LongInt;
LinRHS   : LongInt;
ADiffVars : array[0..49, 0..63] of Char;
ADiffNVars : LongInt;
end;

```

C++ (gecode\_bridge.cpp):

```

struct Constraint {
    int  type;           // ConstraintType
    char var1[64];
    char var2[64];
    int  constant;
    int  lo, hi;
    bool lo_open, hi_open;
    int  set_vals[100];
    int  set_size;
    char lin_vars[20][64];
    int  lin_coefs[20];
    int  lin_nvars;
    int  lin_rhs;
    char adiff_vars[50][64];
    int  adiff_nvars;
};

```

**Nota:** Solo se llenan los campos relevantes al tipo de restriccion (discriminador CType/type).

## TCSPSolution / Solution

Pascal:

```

type
  TCSPSolution = record
    Names   : array[0..49, 0..63] of Char;
    Values  : array[0..49] of LongInt;
    NumVars : LongInt;
  end;

```

C++:

```

struct Solution {
    char names[50][64];
    int  values[50];
    int  num_vars;
};

```

## API Exportada desde C++

La API C exportada es **minimalista y eficiente**, exponiendo solo lo necesario:

### Funciones Principales

#### csp\_create

```
void* csp_create(Variable* vars, int n);
```

**Proposito:** Crear un modelo CSP con N variables.

**Parametros:** - vars: Array de definiciones de variables - n: Numero de variables

**Retorno:** Puntero opaco al modelo (handle)

**Uso Pascal:**

```
var
  Vars: array[0..1] of TCSPVar;
  Model: Pointer;
begin
  Vars[0] := CSPMakeVar('x', 1, 5);
  Vars[1] := CSPMakeVar('y', 0, 10);
  Model := csp_create(@Vars[0], 2);
end;
```

#### csp\_add\_constraint

```
int csp_add_constraint(void* model, Constraint* constraint);
```

**Proposito:** Agregar una restriccion al modelo.

**Parametros:** - model: Handle del modelo - constraint: Puntero a estructura de restriccion

**Retorno:** 1 si OK, 0 si error

**Uso Pascal:**

```
var
  C: TCSPConstraint;
begin
  C := CSPEq('x', 5); // x = 5
  if csp_add_constraint(Model, @C) = 0 then
    WriteLn('Error al agregar restriccion');
end;
```

#### csp\_solve\_first

```
int csp_solve_first(void* model, Solution* sol);
```

**Proposito:** Encontrar la primera solucion.

**Retorno:** 1 si encontro solucion, 0 si no hay

#### csp\_solve\_all

```
int csp_solve_all(void* model, Solution* solutions, int max_sols);
```

**Proposito:** Encontrar todas las soluciones (hasta max\_sols).

**Retorno:** Numero de soluciones encontradas

**Uso Pascal:**

```
var
  Solutions: array[0..99] of TCSPSolution;
  NumSols: Integer;
  i, j: Integer;
begin
  NumSols := csp_solve_all(Model, @Solutions[0], 100);

  for i := 0 to NumSols - 1 do
  begin
    for j := 0 to Solutions[i].NumVars - 1 do
      Write(Format('%s=%d ', [
        PChar(@Solutions[i].Names[j]),
        Solutions[i].Values[j]
      ]));
    WriteLn;
  end;
end;
```

#### csp\_count\_solutions

```
int csp_count_solutions(void* model);
```

**Proposito:** Contar soluciones sin devolverlas (mas eficiente).

**Retorno:** Numero total de soluciones

#### csp\_count\_with\_constraint

```
int csp_count_with_constraint(void* model, Constraint* constraint);
```

**Proposito:** Contar soluciones con una restriccion adicional **sin modificar** el modelo.

**Uso:** Exploracion del espacio de soluciones (usado por GecodeInfo).

#### csp\_debug\_domains

```
int csp_debug_domains(void* model);
```

**Proposito:** Propagar restricciones e imprimir dominios por stderr (diagnostico).

**Retorno:** 0 si FAILED, 1 si OK

**csp\_free**

```
void csp_free(void* model);
```

**Proposito:** Liberar memoria del modelo.

## Tipos de Restricciones Soportadas

El bridge soporta **21 tipos de restricciones** que cubren la mayoría de casos de uso CSP:

### Restricciones Basicas

```
const
CT_EQ    = 0;    // var1 = var2 | const
CT_NEQ   = 1;    // var1 <> var2 | const
CT_LT    = 2;    // var1 < var2 | const
CT_GT    = 3;    // var1 > var2 | const
CT_LE    = 4;    // var1 <= var2 | const
CT_GE    = 5;    // var1 >= var2 | const
```

Ejemplo:

```
C := CSPEq('x', 10);        // x = 10
C := CSPEqVar('x', 'y');    // x = y
C := CSPLt('x', 100);       // x < 100
```

### Restricciones de Dominio

```
const
CT_IN_INTERVAL = 6;    // lo <= var <= hi
CT_IN_SET      = 7;    // var IN {v0, v1, ...}
```

Ejemplo:

```
C := CSPInterval('x', 1, 10);        // x ? [1, 10]
C := CSPInterval('y', 0, 5, True, False); // y ? (0, 5]
C := CSPInSet('color', [1, 3, 5, 7]); // color ? {1,3,5,7}
```

### Restricciones Lineales

```
const
CT_LINEAR_EQ  = 8;    // sum(coef[i]*var[i]) = rhs
CT_LINEAR_LE  = 9;    // sum(coef[i]*var[i]) <= rhs
CT_LINEAR_GE  = 10;   // sum(coef[i]*var[i]) >= rhs
CT_LINEAR_LT  = 11;   // sum(coef[i]*var[i]) < rhs
CT_LINEAR_GT  = 12;   // sum(coef[i]*var[i]) > rhs
CT_LINEAR_NEQ = 13;   // sum(coef[i]*var[i]) <> rhs
```

Ejemplo:

```
// x + y = 10
C := CSPAddEq('x', 'y', 10);

// 2*x - 3*y = 5
C := CSPLinEq2('x', 2, 'y', -3, 5);
```

```
// General:  $3x + 5y - 2z \leq 100$ 
C := CSPLinear(CT_LINEAR_LE,
               ['x', 'y', 'z'],
               [3, 5, -2],
               100);
```

## Restricciones de Valor Absoluto

```
const
CT_ABS_EQ = 14;    //  $abs(var) = const$ 
CT_ABS_LE = 15;    //  $abs(var) \leq const$ 
CT_ABS_GE = 16;    //  $abs(var) \geq const$ 
```

### Ejemplo:

```
C := CSPAbsEq('x', 10);    //  $|x| = 10 \rightarrow x \in \{-10, 10\}$ 
C := CSPAbsLe('y', 5);    //  $|y| \leq 5 \rightarrow y \in [-5, 5]$ 
```

## Restricciones de Distancia

```
const
CT_DIST_EQ = 17;    //  $|var1 - var2| = const$ 
CT_DIST_LE = 18;    //  $|var1 - var2| \leq const$ 
CT_DIST_GE = 19;    //  $|var1 - var2| \geq const$ 
```

### Ejemplo:

```
C := CSPDistEq('x', 'y', 5);    //  $|x - y| = 5$ 
C := CSPDistLe('a', 'b', 10);   //  $|a - b| \leq 10$ 
```

## Restriccion Global

```
const
CT_ALL_DIFF = 20;    //  $all\_different([vars])$ 
```

### Ejemplo:

```
C := CSPAllDiff(['x', 'y', 'z', 'w']);
// Todas las variables deben tener valores distintos
```

# Compilacion Monolitica

El sistema soporta **compilacion monolitica** para crear ejecutables standalone sin dependencias externas.

## Modos de Compilacion

### Modo Estatico

**Requisitos:** - Gecode compilado como biblioteca estatica (.a) - Variable `GECODE_HOME` apuntando a instalacion estatica

**Resultado:** - Ejecutable de ~8-12 MB - **Sin dependencias** de `libgecode*.so` - Solo depende de `libc/libm` del sistema - Completamente portable

### Modo Dinamico

**Requisitos:** - Gecode instalado del sistema (`/usr/lib/libgecode*.so`)

**Resultado:** - Ejecutable de ~500 KB - Requiere `libgecode*.so` en runtime - Mas pequeno pero menos portable

## Script `build_monolithic.sh`

**Ubicacion:** `/home/rodo/gecode/scripts/build_monolithic.sh`

**Uso:**

```
# Compilar programa especifico
./scripts/build_monolithic.sh src/TestGecodeBridge.pas

# Compilar con GECODE_HOME personalizado
GECODE_HOME=$HOME/gecode-static ./scripts/build_monolithic.sh src/TestGecodeBridge.pas
```

## Proceso de Compilacion

El script realiza **3** pasos:

### Paso 1: Compilar Bridge C++

```
g++ -c src/gecode_bridge.cpp \
    -o obj/gecode_bridge.o \
    -I$GECODE_HOME/include \
    -std=c++17 \
    -O2 \
    -DNDEBUG \
    -fvisibility=hidden \
    -ffunction-sections \
    -fdata-sections \
    -fno-stack-protector
```

**Flags importantes:** - `-fvisibility=hidden`: Oculta simbolos internos (solo exporta `csp_*`) - `-ffunction-sections`: Permite garbage collection de secciones no usadas - `-fdata-sections`: Igual para datos

## Paso 2: Compilar Pascal

```
fpc TestGecodeBridge.pas -O2 -Cn -FU obj/ -Fu obj/ -Fu src/
```

**Genera:** - `link*.res`: Script de linkeo con lista de objetos - `obj/*.o`: Objetos Pascal compilados - `obj/*.ppu`: Units compiladas

**Post-procesamiento:**

```
# Ajustar paths en link*.res
sed -i "s|^([A-Za-z][A-Za-z0-9_]*\.o\)$|obj/\1|g" link*.res
```

## Paso 3: Linkeo Final

**Modo estatico:**

```
/usr/bin/ld.bfd -b elf64-x86-64 -m elf_x86_64 -s \
-o bin/TestGecodeBridge \
-T link*.res -e _start \
--dynamic-linker /lib64/ld-linux-x86-64.so.2 \
--gc-sections \
-L$GECODE_HOME/lib \
-Bstatic \
-lgecodeminimodel \
-lgecodeint \
-lgecodesearch \
-lgecodekernel \
-lgecodesupport \
-Bdynamic \
-lstdc++ -lgcc_s -lc
```

**Flags importantes:** - `-Bstatic`: Linkeo estatico para Gecode - `-Bdynamic`: Linkeo dinamico para `libstdc++/libc` - `--gc-sections`: Eliminar secciones no usadas - `-s`: Strip simbolos debug

## Verificacion

```
# Ver tamano
ls -lh bin/TestGecodeBridge
# ~8-12 MB en modo estatico

# Ver dependencias
ldd bin/TestGecodeBridge
# Solo debe mostrar:
#   linux-vdso.so.1
#   libc.so.6
#   libm.so.6
```

```
# libstdc++.so.6 (si no se usa -static-libstdc++)  
# /lib64/ld-linux-x86-64.so.2  
  
# Ver simbolos exportados del bridge  
nm -D bin/TestGecodeBridge | grep " T csp_"  
# Debe listar: csp_create, csp_add_constraint, etc.
```

## Programas de Prueba

El proyecto incluye **4 programas de prueba** que demuestran el uso del bridge:

### 1. TestGecodeBridge

**Proposito:** Resolver CSP completo desde JSON usando el motor Gecode.

**Ubicacion:** /home/rodo/gecode/src/TestGecodeBridge.pas

**Uso:**

```
# Resolver archivo especifico
./bin/TestGecodeBridge grafo.json

# Correr todos los tests
./bin/TestGecodeBridge
```

**Funcionalidad:** - Lee grafo JSON (salida de JsonToGraph) - Traduce variables y restricciones a modelo Gecode - Ejecuta busqueda DFS - Analiza espacio de soluciones - Retorna JSON con analisis

**Salida:**

```
{
  "status": "ok",
  "total_solutions": 252,
  "analysis": [
    {
      "name": "ZONA",
      "type": "set",
      "if_fixed": [
        { "value": "NORTE", "solutions": 126 },
        { "value": "SUR", "solutions": 126 }
      ]
    }
  ]
}
```

### 2. GecodeInfo

**Proposito:** Analizar el espacio de soluciones sin enumerar todas.

**Ubicacion:** /home/rodo/gecode/src/GecodeInfo.pas

**Uso:**

```
./bin/GecodeInfo grafo.json
```

**Funcionalidad:** - Para cada variable no-determinada - Calcula cuantas soluciones existen si se fija a cada valor posible - Genera mapa de distribucion del espacio de busqueda

**Salida:**

```
{
  "status": "ok",
  "total_solutions": 100,
  "analysis": [
    {
      "name": "X",
      "type": "integer",
      "if_fixed": [
        { "value": 1, "solutions": 10 },
        { "value": 2, "solutions": 20 },
        { "value": 3, "solutions": 30 },
        { "value": 4, "solutions": 25 },
        { "value": 5, "solutions": 15 }
      ]
    }
  ]
}
```

**Diferencia con TestGecodeBridge:** - GecodeInfo: **analisis** del espacio - TestGecodeBridge: **enumeracion** + analisis

### 3. TestComplejo

**Proposito:** Demostracion de dos problemas CSP clasicos usando el bridge directamente desde Pascal (sin JSON).

**Ubicacion:** /home/rodo/gecode/src/TestComplejo.pas

**Uso:**

```
./bin/TestComplejo
```

#### Problema 1: SEND + MORE = MONEY

**Descripcion:** Criptoaritmetica clasica.

**Restricciones:**

```
// 8 variables: S, E, N, D, M, O, R, Y
// Dominios: [0..9] excepto S, M ? [1..9]

// All different
CSPAllDiff(['S', 'E', 'N', 'D', 'M', 'O', 'R', 'Y'])

// Ecuacion: SEND + MORE = MONEY
// 1000*S + 100*E + 10*N + D +
// 1000*M + 100*O + 10*R + E =
// 10000*M + 1000*O + 100*N + 10*E + Y

// Simplificado:
CSPLinear(CT_LINEAR_EQ,
```

```
['S', 'E', 'N', 'D', 'M', 'O', 'R', 'Y'],
[1000, 91, -90, 1, -9000, -900, 10, -1],
0)
```

**Solucion unica:**

```
SEND  = 9567
MORE  = 1085
MONEY = 10652
```

## Problema 2: Cuadrado Magico 3x3

**Descripcion:** Colocar numeros 1..9 en cuadrícula 3x3 donde todas las filas, columnas y diagonales sumen 15.

**Restricciones:**

```
// 9 variables: C11, C12, C13, C21, C22, C23, C31, C32, C33
// Todas en [1..9] y distintas
CSPAllDiff([...])

// Filas suman 15
CSPLinear(CT_LINEAR_EQ, ['C11', 'C12', 'C13'], [1,1,1], 15)
CSPLinear(CT_LINEAR_EQ, ['C21', 'C22', 'C23'], [1,1,1], 15)
CSPLinear(CT_LINEAR_EQ, ['C31', 'C32', 'C33'], [1,1,1], 15)

// Columnas suman 15
CSPLinear(CT_LINEAR_EQ, ['C11', 'C21', 'C31'], [1,1,1], 15)
CSPLinear(CT_LINEAR_EQ, ['C12', 'C22', 'C32'], [1,1,1], 15)
CSPLinear(CT_LINEAR_EQ, ['C13', 'C23', 'C33'], [1,1,1], 15)

// Diagonales suman 15
CSPLinear(CT_LINEAR_EQ, ['C11', 'C22', 'C33'], [1,1,1], 15)
CSPLinear(CT_LINEAR_EQ, ['C13', 'C22', 'C31'], [1,1,1], 15)
```

**Soluciones:** 8 (rotaciones y reflexiones del cuadrado base)

## 4. TestIntervalos

**Proposito:** Resolver sistema de restricciones lineales con dominios continuos discretizados.

**Ubicacion:** /home/rodo/gecode/src/TestIntervalos.pas

**Uso:**

```
./bin/TestIntervalos
```

**Sistema:**

**Variables:**

```
x ? [-2, 5] (discretizado en paso 0.1)
y ? [-1, 6] (discretizado en paso 0.1)
```

Restricciones:

$$5x + 3y \leq [1, 3]$$

$$6x + 10y \leq [10, 50]$$

Discretizacion:

```
// Multiplicar por 10 para evitar punto flotante
```

```
VarX := CSPMakeVar('x', -20, 50); // x x 10
```

```
VarY := CSPMakeVar('y', -10, 60); // y x 10
```

```
// Restriccion: 5x + 3y ≤ [1, 3]
```

```
// -> (5x(x/10) + 3x(y/10)) x 10 ≤ [10, 30]
```

```
// -> 5x + 3y ≤ [10, 30]
```

**Salida:** - Estadísticas de region factible - Plot ASCII de la region - Lista de puntos solucion

## Ejemplos de Uso

### Ejemplo 1: Resolver CSP Simple

```
program EjemploSimple;

uses UGecodeBridge;

var
  Model: Pointer;
  Vars: array[0..1] of TCSPVar;
  C: TCSPConstraint;
  Sol: TCSPSolution;
  i: Integer;

begin
  // Definir variables
  Vars[0] := CSPMakeVar('x', 1, 5);
  Vars[1] := CSPMakeVar('y', 1, 10);

  // Crear modelo
  Model := csp_create(@Vars[0], 2);

  // Restriccion:  $x + y = 10$ 
  C := CSPAddEq('x', 'y', 10);
  csp_add_constraint(Model, @C);

  // Resolver (primera solucion)
  if csp_solve_first(Model, @Sol) = 1 then
  begin
    WriteLn('Solucion:');
    for i := 0 to Sol.NumVars - 1 do
      WriteLn('  ', PChar(@Sol.Names[i]), ' = ', Sol.Values[i]);
    end
  else
    WriteLn('Sin solucion');

  // Liberar
  csp_free(Model);
end.
```

### Ejemplo 2: Todas las Soluciones

```
program TodasSoluciones;

uses UGecodeBridge;

var
```

```

Model: Pointer;
Vars: array[0..2] of TCSPVar;
C: TCSPConstraint;
Solutions: array[0..99] of TCSPSolution;
NumSols, i, j: Integer;

begin
    // Variables: x, y, z ? [1, 3]
    Vars[0] := CSPMakeVar('x', 1, 3);
    Vars[1] := CSPMakeVar('y', 1, 3);
    Vars[2] := CSPMakeVar('z', 1, 3);

    Model := csp_create(@Vars[0], 3);

    // Todas diferentes
    C := CSPAllDiff(['x', 'y', 'z']);
    csp_add_constraint(Model, @C);

    // x + y + z = 6
    C := CSPLinear(CT_LINEAR_EQ,
                  ['x', 'y', 'z'],
                  [1, 1, 1],
                  6);
    csp_add_constraint(Model, @C);

    // Resolver todas
    NumSols := csp_solve_all(Model, @Solutions[0], 100);

    WriteLn('Soluciones: ', NumSols);
    for i := 0 to NumSols - 1 do
    begin
        Write(Format('%2d. ', [i + 1]));
        for j := 0 to Solutions[i].NumVars - 1 do
            Write(Format('%s=%d ', [
                PChar(@Solutions[i].Names[j]),
                Solutions[i].Values[j]
            ]));
        WriteLn;
    end;

    csp_free(Model);
end.

```

### Ejemplo 3: Exploracion del Espacio

```

program ExploracionEspacio;

```

```

uses UGecodeBridge;

var
  Model: Pointer;
  Vars: array[0..1] of TCSPVar;
  C: TCSPConstraint;
  Total, Count, Val: Integer;

begin
  // x, y ? [1, 10]
  Vars[0] := CSPMakeVar('x', 1, 10);
  Vars[1] := CSPMakeVar('y', 1, 10);

  Model := csp_create(@Vars[0], 2);

  // x + y <= 15
  C := CSPAddLe('x', 'y', 15);
  csp_add_constraint(Model, @C);

  // Total de soluciones
  Total := csp_count_solutions(Model);
  WriteLn('Total: ', Total, ' soluciones');
  WriteLn;

  // ?Cuantas soluciones si x = 1? x = 2? etc.
  WriteLn('Análisis:');
  for Val := 1 to 10 do
    begin
      C := CSPEq('x', Val);
      Count := csp_count_with_constraint(Model, @C);
      WriteLn(' Si x = ', Val, ' -> ', Count, ' soluciones');
    end;

  csp_free(Model);
end.

```

# Resumen de la Integracion

## Ventajas de esta Arquitectura

1. **Separacion de responsabilidades**
  - Pascal: Logica de negocio, I/O, pipeline
  - C++: Solo interfaz a Gecode
  - C: Funciones matematicas low-level
2. **Ejecutables monoliticos**
  - Un solo archivo standalone
  - Sin DLLs externas
  - Portable entre sistemas
3. **API minimalista**
  - Solo 8 funciones exportadas
  - Estructuras simples (POD)
  - Sin overhead de runtime
4. **Eficiencia**
  - Garbage collection de simbolos no usados
  - Linkeo estatico de Gecode
  - Optimizaciones de compilador
5. **Mantenibilidad**
  - Bridge C++ aislado en un solo archivo
  - Interface Pascal clara y tipada
  - Helpers Pascal para construccion facil

## Limitaciones

1. **Variables solo enteras**
  - No soporta float directamente (se discretiza)
  - No soporta set variables de Gecode (se mapean a int)
2. **Estructuras de tamano fijo**
  - Arrays con limites predefinidos
  - No dinamicos para compatibilidad C
3. **Linkeo manual**
  - Requiere script de compilacion especifico
  - No usa build system estandar

## Flujo Completo

1. Escribir programa Pascal
  - Usar UGecodeBridge
  - Crear variables con CSPMakeVar
  - Crear restricciones con CSP\* helpers
  - Llamar csp\_create, csp\_add\_constraint, csp\_solve\_\*
2. Compilar con build\_monolithic.sh
  - Compila gecode\_bridge.cpp -> .o
  - Compila Pascal -> .o
  - Linkea todo + Gecode estatico

### 3. Ejecutar

- Ejecutable standalone
- Sin dependencias
- Totalmente portable

---

**Conclusion:** La integracion Pascal/C++/Gecode permite combinar la **productividad de Pascal** para el pipeline con el **poder de Gecode** para resolucion CSP, generando **ejecutables eficientes y portables**.